Python Data Structures

Lists • Tuples • Dictionaries • Sets

Week 1 - Day 2

AI/ML/DL Professional Course

R.A: MUHAMMAD HUZIFA

Learning Objectives

1. Master Python Lists and their methods (append, insert, remove, sort)
2. Understand Tuples and their immutability
3. Work with Dictionaries for key-value data storage
4. Use Sets for unique collections and set operations
5. Choose the right data structure for specific problems
6. Apply data structures to solve real-world challenges

What are Data Structures?

Data Structures are specialized formats for organizing, storing, and managing data efficiently. They allow us to work with collections of data instead of individual variables.

Why Do We Need Data Structures?

- Organize large amounts of data efficiently
- Fast data access and manipulation
- Simplify complex programming tasks
- Optimize memory usage

Python's Built-in Data Structures:

List	Tuple	Dictionary	Set
Ordered, mutable	Ordered, immutable	Key-value pairs	Unique, unordered

Python Lists

What is a List?

A List is an ordered, mutable (changeable) collection that can store multiple items of different types. Lists are created using square brackets [ ].

Key Characteristics:

✓ Ordered - items maintain insertion order
✓ Mutable - can add, remove, or change items
✓ Allows duplicates - same value can appear multiple times
✓ Indexed - access items by position (starting from 0)

Creating Lists:

```
# Empty list
empty = []
# List of numbers
numbers = [1, 2, 3, 4, 5]
# List of strings
fruits = ["apple", "banana", "cherry"]
# Mixed data types
mixed = [1, "hello", 3.14, True]
```

List Indexing & Slicing

Understanding List Indexing:

```
List:     ['A', 'B', 'C', 'D', 'E']
Index:     0    1    2    3    4
Negative: -5   -4   -3   -2   -1
```

Examples:

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

# Accessing by index
print(fruits[0])      # apple (first item)
print(fruits[2])      # cherry (third item)
print(fruits[-1])     # elderberry (last item)
print(fruits[-2])     # date (second from end)

# Slicing [start:end:step]
print(fruits[1:4])    # ['banana', 'cherry', 'date']
print(fruits[:3])     # ['apple', 'banana', 'cherry']
print(fruits[2:])     # ['cherry', 'date', 'elderberry']
print(fruits[::2])    # ['apple', 'cherry', 'elderberry'] (every 2nd)
```

Essential List Methods

`append(item)`
Add item to end → list.append(4)

`insert(i, item)`
Insert at position → list.insert(1, 'x')

`remove(item)`
Remove first occurrence → list.remove('a')

`pop(i)`
Remove & return item → list.pop(0)

`clear()`
Remove all items → list.clear()

`sort()`
Sort in place → list.sort()

`reverse()`
Reverse order → list.reverse()

`index(item)`
Find item position → list.index('x')

`count(item)`
Count occurrences → list.count(5)

`extend(list2)`
Add another list → list.extend([7,8])

Python Tuples

What is a Tuple?

A Tuple is an ordered, immutable (unchangeable) collection. Once created, you cannot modify its contents. Tuples are created using parentheses ( ).

List vs Tuple:

Syntax	[ ]	( )
Mutable	Yes ✓	No ✗
Speed	Slower	Faster
Use Case	Data changes	Fixed data

Creating Tuples:

```
# Empty tuple
empty = ()
# Tuple with values
coordinates = (10, 20)
rgb = (255, 128, 0)
data = ("Ahmed", 25, "Lahore")  # Mixed types
```

Python Dictionaries

What is a Dictionary?

A Dictionary is an unordered collection of key-value pairs. Each key must be unique and is used to access its corresponding value. Dictionaries are created using curly braces { }.

Key Characteristics:

✓ Key-Value Pairs - data stored as key: value
✓ Mutable - can add, remove, or modify entries
✓ Unique Keys - each key appears only once
✓ Fast Lookup - retrieve values by key instantly

Creating Dictionaries:

```
# Empty dictionary
empty = {}

# Student information
student = {
    "name": "Ahmed Ali",
    "age": 25,
    "city": "Lahore",
    "courses": ["AI", "ML", "DL"]
}

# Accessing values
print(student["name"])     # Ahmed Ali
print(student["age"])      # 25
```

Dictionary Methods

keys()

Get all keys → dict.keys()

values()

Get all values → dict.values()

items()

Get key-value pairs → dict.items()

get(key)

Safe key access → dict.get('name')

update(dict2)

Merge dictionaries → dict.update({...})

pop(key)

Remove & return → dict.pop('age')

clear()

Remove all items → dict.clear()

copy()

Create shallow copy → new = dict.copy()

Practical Example:

```
# Student grades management
grades = {"Math": 85, "Physics": 90, "Chemistry": 88}
# Add new subject
grades["Biology"] = 92
# Loop through dictionary
for subject, score in grades.items():
    print(f"{subject}: {score}")
```

Python Sets

What is a Set?

A Set is an unordered collection of unique items. Duplicates are automatically removed. Sets are created using curly braces { } or set().

Key Characteristics:

✓ Unique Items - no duplicates allowed

✓ Unordered - items have no specific order

✓ Mutable - can add or remove items

✓ Fast Membership Testing - check if item exists quickly

Creating Sets:

```
# Empty set (must use set())
empty = set()
# Set with values
numbers = {1, 2, 3, 4, 5}
fruits = {"apple", "banana", "cherry"}
# Duplicates automatically removed
nums = {1, 2, 2, 3, 3, 3}
print(nums)  # Output: {1, 2, 3}
# Check membership
print(2 in nums)  # True
```

Set Operations

Mathematical Set Operations:

Union `|`	All items from both sets `A | B or A.union(B)`
Intersection `&`	Common items only `A & B or A.intersection(B)`
Difference	Items in A but not in B `A - B or A.difference(B)`
Symmetric Diff `^`	Items in A or B, not both `A ^ B or A.symmetric_difference(B)`

Example:

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print(A | B)   # {1, 2, 3, 4, 5, 6, 7, 8}  (Union)
print(A & B)   # {4, 5}                   (Intersection)
print(A - B)   # {1, 2, 3}                (Difference)
print(A ^ B)   # {1, 2, 3, 6, 7, 8}       (Symmetric Diff)
```

Choosing the Right Data Structure

When to Use Each:

List

- Ordered collection needed
- Duplicates are allowed
- Frequent item changes
- Access by index position

Example: Shopping cart, Todo list

Tuple

- Data shouldn't change
- Faster than lists
- Use as dictionary keys
- Return multiple values

Example: Coordinates (x,y), RGB colors

Dictionary

- Key-value relationships
- Fast lookup by key
- Named access to data
- Complex data structures

Example: Student records, Config settings

Set

- Unique items only
- Membership testing
- Mathematical operations
- Remove duplicates

Example: Unique visitors, Tag system

List Comprehension - Advanced Technique

What is List Comprehension?

A concise way to create lists based on existing lists or ranges. It's more readable and faster than traditional loops.

Traditional Way vs List Comprehension:

Traditional Loop

```
# Create squares
squares = []
for i in range(10):
    squares.append(i ** 2)
```

List Comprehension

```
# Same result, one line!
squares = [i ** 2 for i in range(10)]


# Clean and efficient!
```

More Examples:

```
# Even numbers
evens = [x for x in range(20) if x % 2 == 0]
# Convert to uppercase
words = ["hello", "world", "python"]
upper = [w.upper() for w in words]
# Nested comprehension
matrix = [[i*j for j in range(3)] for i in range(3)]
```

Nested Data Structures

Combining Data Structures:

You can combine data structures to create complex data models - lists inside dictionaries, dictionaries inside lists, etc.

Example 1: List of Dictionaries (Students Database)

```
students = [
    {"name": "Ahmed", "age": 25, "grades": [85, 90, 88]},
    {"name": "Fatima", "age": 23, "grades": [92, 88, 95]},
    {"name": "Ali", "age": 24, "grades": [78, 85, 80]}
]

# Access data
print(students[0]["name"])        # Ahmed
print(students[1]["grades"][0])   # 92
```

Example 2: Dictionary of Lists (Course Enrollments)

```
courses = {
    "AI": ["Ahmed", "Fatima", "Ali"],
    "ML": ["Ahmed", "Sara"],
    "DL": ["Fatima", "Ali", "Sara"]
}
# Check enrollment
print("AI" in courses)           # True
print(len(courses["ML"]))        # 2
```

Data Structures in AI & Machine Learning

Why This Matters for AI/ML:

Lists

Store training data, feature vectors, predictions

```
features = [height, weight, age, salary]
```

Dictionaries

Hyperparameters, model configurations, results

```
config = {'learning_rate': 0.01, 'epochs':
100}
```

Tuples

Fixed data like image dimensions, coordinates

```
image_size = (256, 256, 3)  # width, height, channels
```

Sets

Unique labels, vocabulary in NLP, feature selection

```
unique_classes = {'cat', 'dog', 'bird'}
```

Note:

💡 Mastering data structures is essential for AI/ML - NumPy and Pandas build on these concepts!

Common Mistakes & How to Avoid Them

❌ Modifying list while iterating

Wrong:

```
for item in list:
    list.remove(item)
```

Correct:

```
for item in list.copy():
    list.remove(item)
```

❌ Forgetting tuples are immutable

Wrong:

```
coords = (10, 20)
coords[0] = 30  # Error!
```

Correct:

```
coords = (10, 20)
coords = (30, 20)  # New tuple
```

❌ Using mutable default arguments

Wrong:

```
def func(items=[]):
    items.append(1)
```

Correct:

```
def func(items=None):
    if items is None:
        items = []
```

Today's Practice Tasks

Complete These Exercises:

Task 21

List Operations

Practice list methods and indexing

⏱ 20 mins

Task 22

Tuple Exercises

Work with immutable sequences

⏱ 15 mins

Task 23

String Manipulation

Advanced string operations

⏱ 20 mins

Task 24

Set Operations

Union, intersection, difference

⏱ 15 mins

Real-World Applications

E-Commerce Shopping Cart

```
cart = [
    {'item': 'Laptop', 'price': 80000, 'qty':
1},
    {'item': 'Mouse', 'price': 1500, 'qty': 2}
]
```

Social Media Tags System

Uses: Set

```
post_tags = {'python', 'AI', 'ML',
'coding'}
new_tags = {'AI', 'deep_learning'}
all_tags = post_tags | new_tags
```

Student Grade Management

Uses: Dictionary

```
grades = {
    'Ahmed': {'Math': 85, 'Physics': 90},
    'Fatima': {'Math': 92, 'Physics': 88}
}
```

GPS Coordinates Tracking

Uses: List of Tuples

```
route = [
    (31.5204, 74.3587),  # Lahore
    (33.6844, 73.0479),  # Islamabad
]
```

Day 2 Summary - What We Learned

Lists

- Ordered, mutable collections with indexing
- Methods: append, insert, remove, sort, reverse
- List comprehension for efficient creation

Tuples

- Ordered, immutable collections
- Faster than lists, used for fixed data
- Can be used as dictionary keys

Dictionaries

- Key-value pairs for structured data
- Fast lookup, insertion, and deletion
- Methods: keys(), values(), items(), get()

Sets

- Unordered collections of unique items

Homework & Next Class

📚 Homework for Tonight:

1. Complete Tasks 21-25 from Annexure-I (all data structure exercises)
2. Practice list comprehension with 5 different examples
3. Create a student management system using dictionaries
4. Solve all challenges in the Jupyter Notebook